Django Database & ORM — Complete Notes A to Z

Django 6.0 | Beginner to Advanced | English Explanation with Python Examples

Part 1: Introduction to Databases

1. What Is a Database?

A database is an organized system for storing, retrieving, updating, and deleting data.

Examples:

- Student names and email addresses
- Course information
- Student course enrollments
- Examination results
- Product and order information

Common database systems include:

- PostgreSQL
- MySQL
- SQLite
- Oracle

2. What Is a Relational Database?

A relational database stores data in tables consisting of rows and columns.

Example: Student table

id	name	email
1	Omar	omar@example.com
2	Rahim	rahim@example.com

- Row: One record.
- Column: One field.
- Primary key: Uniquely identifies a row.

- Foreign key: Connects related records across tables.
- Index: Helps the database find records efficiently.

3. What Is an ORM?

ORM stands for Object-Relational Mapping.

Django ORM allows you to interact with a relational database using Python instead of writing SQL for every operation.

SQL example:

```
SELECT * FROM students WHERE age > 18;
```

Django ORM equivalent:

```
Student.objects.filter(age__gt=18)
```

Advantages:

- Less repetitive SQL.
- Python-based database operations.
- Model-based data organization.
- Database portability, with some backend-specific limitations.
- Query composition and lazy evaluation.
- Safer parameterized queries when used correctly.

Important: ORM does not eliminate the need to understand SQL, indexes, joins, transactions, and database performance.

Part 2: Django Database Configuration

4. Database Configuration in settings.py

Django stores database configuration in the DATABASES setting.

SQLite example:

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "db.sqlite3",
    }
}
```

PostgreSQL example:

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.postgresql",
        "NAME": "school_db",
        "USER": "postgres",
        "PASSWORD": "your_password",
        "HOST": "localhost",
        "PORT": "5432",
    }
}
```

Install the appropriate PostgreSQL driver according to the Django 6.0 installation documentation.

Security best practice:

- Do not hardcode production database passwords in source code.
- Use environment variables or a secret-management system.
- Give the database user only the permissions it needs.

5. Essential Database Commands

Create a Django project:

```
django-admin startproject config .
```

Create an application:

```
python manage.py startapp students
```

Apply existing migrations:

```
python manage.py migrate
```

Create migration files:

```
python manage.py makemigrations
```

Open the interactive shell:

```
python manage.py shell
```

Check for common project problems:

```
python manage.py check
```

Part 3: Django Models

6. What Is a Model?

A Django model is a Python class that describes the structure and behavior of data stored in a database.

Example:

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField(unique=True)
    age = models.PositiveIntegerField()
    created_at = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return self.name
```

By default, Django creates a primary-key field automatically unless the model specifies its own primary key.

7. Common Model Fields

```
models.CharField(max_length=100)
models.TextField()
models.IntegerField()
models.PositiveIntegerField()
models.BooleanField(default=True)
models.EmailField()
models.URLField()
models.DateField()
models.DateTimeField()
models.DecimalField(max_digits=10, decimal_places=2)
models.FloatField()
models.UUIDField()
models.JSONField()
models.FileField(upload_to="files/")
models.ImageField(upload_to="images/")
```

Use DecimalField for monetary amounts when decimal precision is required.

8. Important Model Field Options

- `null=True` : Allows the database column to store SQL NULL.
- `blank=True` : Allows the field to be empty during validation.

- `default=value`: Provides a default value.
- `unique=True`: Requires unique values.
- `db_index=True`: Requests a database index.
- `primary_key=True`: Defines the primary key.
- `choices=...`: Defines a set of allowed choices for model/form validation.
- `help_text="..."`: Provides descriptive help text.
- `verbose_name="..."`: Sets a human-readable field name.

Example:

```
class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.DecimalField(
        max_digits=10,
        decimal_places=2
    )
    description = models.TextField(blank=True)
    is_available = models.BooleanField(default=True)
```

Remember:

- `null` concerns database storage.
- `blank` concerns validation.
- `choices` does not automatically guarantee a database-level restriction in every configuration.
- Use appropriate database constraints when integrity must be enforced at the database level.

9. Model Meta Options

```
class Student(models.Model):
    name = models.CharField(max_length=100)

    class Meta:
        ordering = ["name"]
        db_table = "students"
        verbose_name = "Student"
        verbose_name_plural = "Students"
```

Useful Meta options include:

- `ordering`
- `db_table`
- `verbose_name`

- `verbose_name_plural`
- `constraints`
- `indexes`
- `unique_together` (legacy; prefer `UniqueConstraint` for new work)

Example constraint:

```
class Product(models.Model):
    name = models.CharField(max_length=100)
    price = models.DecimalField(
        max_digits=10,
        decimal_places=2
    )

    class Meta:
        constraints = [
            models.CheckConstraint(
                condition=models.Q(price__gte=0),
                name="product_price_nonnegative",
            )
        ]
```

Check the supported constraint API and backend behavior in the Django 6.0 documentation.

Part 4: Migrations

10. What Are Migrations?

Migrations are version-controlled instructions that describe changes to database schema.

Common workflow:

```
python manage.py makemigrations
python manage.py migrate
```

Other useful commands:

```
python manage.py showmigrations
python manage.py sqlmigrate students 0001
python manage.py migrate students zero
```

The last command can reverse migrations for that application; use it carefully, especially when data may be affected.

Important:

- `makemigrations` creates migration files.
- `migrate` applies migrations to the database.
- Commit migration files to version control.
- Review migrations before deploying.
- Schema changes can affect existing data and running applications.
- Migration rollback does not guarantee that deleted or transformed data can be recovered.

Part 5: CRUD Operations

CRUD means Create, Read, Update, and Delete.

Assume this model exists:

```
class Student(models.Model):
    name = models.CharField(max_length=100)
    email = models.EmailField(unique=True)
    age = models.PositiveIntegerField()
```

11. Create

Create an object:

```
student = Student.objects.create(
    name="Omar",
    email="omar@example.com",
    age=22
)
```

Alternatively:

```
student = Student(
    name="Rahim",
    email="rahim@example.com",
    age=23
)
student.save()
```

12. Read

Retrieve all records:

```
students = Student.objects.all()
```

Retrieve one record:

```
student = Student.objects.get(id=1)
```

Retrieve records matching a condition:

```
students = Student.objects.filter(age=22)
```

Get the first matching record:

```
student = Student.objects.filter(age=22).first()
```

Check whether matching records exist:

```
exists = Student.objects.filter(age=22).exists()
```

Count matching records:

```
count = Student.objects.filter(age=22).count()
```

Important:

- `get()` raises `Student.DoesNotExist` when no record matches.
- `get()` raises `Student.MultipleObjectsReturned` when multiple records match.
- `filter()` returns a QuerySet, even when no records match.
- `first()` returns the first matching object or `None`.

13. Update

Update one object:

```
student = Student.objects.get(id=1)
student.name = "Updated Name"
student.save()
```

Update matching rows directly:

```
Student.objects.filter(id=1).update(name="Updated Name")
```

The QuerySet `update()` method performs a direct database update and does not call each object's `save()` method or emit normal save signals.

14. Delete

Delete one object:

```
student = Student.objects.get(id=1)
student.delete()
```

Delete matching records:

```
Student.objects.filter(age__lt=18).delete()
```

Be careful: deletion may cascade to related records depending on relationship configuration. Test destructive operations before using them in production.

Part 6: QuerySets

15. What Is a QuerySet?

A QuerySet represents a collection of database objects returned by a Django query.
Example:

```
students = Student.objects.filter(age__gte=18)
```

QuerySets are generally lazy: constructing one does not immediately execute the database query.
The query is usually evaluated when its results are needed.
Examples of evaluation triggers:

- Iterating over the QuerySet.
- Converting it to a list.
- Calling `len()` or `bool()` in applicable situations.
- Accessing an object by indexing in a way that requires a query.
- Calling methods such as `list()` on it.

Examples:

```
students = Student.objects.all()
students = students.filter(age__gte=18)
students = students.order_by("name")
```

These operations can build a query without immediately fetching all results.

16. Chaining QuerySets

```
students = Student.objects.filter(
    age__gte=18
).exclude(
    name__startswith="A"
).order_by(
    "-age"
)
```

This retrieves students aged at least 18, excluding names starting with A, ordered by descending age.

17. Useful QuerySet Methods

```
Student.objects.all()
Student.objects.filter(age=20)
Student.objects.exclude(age=20)
Student.objects.get(id=1)
Student.objects.first()
Student.objects.last()
Student.objects.count()
Student.objects.exists()
Student.objects.order_by("name")
Student.objects.values("name", "email")
Student.objects.values_list("name", flat=True)
Student.objects.distinct()
```

Use `values()` or `values_list()` when you need specific field values rather than full model instances.

Part 7: Field Lookups

18. Common Lookups

```
Student.objects.filter(age__gt=18)
Student.objects.filter(age__gte=18)
Student.objects.filter(age__lt=25)
Student.objects.filter(age__lte=25)
Student.objects.filter(name__exact="Omar")
Student.objects.filter(name__iexact="omar")
Student.objects.filter(name__contains="om")
Student.objects.filter(name__icontains="om")
Student.objects.filter(name__startswith="O")
Student.objects.filter(name__endswith="r")
Student.objects.filter(age__in=[18, 20, 22])
Student.objects.filter(age__range=(18, 25))
Student.objects.filter(email__isnull=True)
```

Common lookup meanings:

- `exact` : Exact match.
- `iexact` : Case-insensitive exact match.

- `contains` : Contains a substring.
- `icontains` : Case-insensitive substring match.
- `gt` : Greater than.
- `gte` : Greater than or equal to.
- `lt` : Less than.
- `lte` : Less than or equal to.
- `in` : Matches one of the supplied values.
- `range` : Inclusive range.
- `isnull` : Checks whether a field is NULL.

String matching and case sensitivity may depend on the database backend and its collation.

19. Q Objects

Use Q objects for complex conditions involving OR, AND, or NOT.

```
from django.db.models import Q
```

OR:

```
Student.objects.filter(
    Q(age=18) | Q(age=20)
)
```

AND:

```
Student.objects.filter(
    Q(age__gte=18) & Q(name__startswith="O")
)
```

NOT:

```
Student.objects.filter(~Q(age=18))
```

Part 8: Relationships

20. ForeignKey — Many-to-One

Many students can belong to one department.

```
class Department(models.Model):
    name = models.CharField(max_length=100)

    def __str__(self):
```

```
        return self.name


class Student(models.Model):
    name = models.CharField(max_length=100)
    department = models.ForeignKey(
        Department,
        on_delete=models.CASCADE,
        related_name="students"
    )
```

`on_delete=models.CASCADE` tells Django how to handle related objects when the referenced object is deleted through Django's ORM behavior. It is not identical to configuring a database-level SQL `ON DELETE CASCADE` action in every setup.
Common `on_delete` options:

- `CASCADE`
- `PROTECT`
- `RESTRICT`
- `SET_NULL`
- `SET_DEFAULT`
- `DO_NOTHING`

For `SET_NULL`, the ForeignKey must allow NULL values.

21. One-to-One Relationship

Example: One user has one profile.

```
from django.conf import settings

class Profile(models.Model):
    user = models.OneToOneField(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE
    )
    bio = models.TextField(blank=True)
```

Using `settings.AUTH_USER_MODEL` helps support custom user models.

22. Many-to-Many Relationship

Example: Students can enroll in multiple courses, and each course can have multiple students.

```
class Course(models.Model):
    title = models.CharField(max_length=100)
```

```
class Student(models.Model):
    name = models.CharField(max_length=100)
    courses = models.ManyToManyField(
        Course,
        related_name="students"
    )
```

Example usage:

```
student.courses.add(course)
student.courses.remove(course)
student.courses.clear()
```

For additional enrollment fields, use an explicit intermediary model with `through`.

23. Querying Relationships

Filter through a ForeignKey:

```
Student.objects.filter(department__name="Computer Science")
```

Access the related object:

```
student.department.name
```

Access reverse relationships:

```
department.students.all()
```

Filter through a Many-to-Many relationship:

```
Student.objects.filter(courses__title="Python")
```

Part 9: select_related and prefetch_related

24. select_related()

Use `select_related()` for ForeignKey and OneToOne relationships. It uses SQL joins to retrieve related objects in the same query.

```
students = Student.objects.select_related(
    "department"
)

for student in students:
    print(student.department.name)
```

This can prevent an extra query for each student's department.

25. prefetch_related()

Use `prefetch_related()` for ManyToMany and reverse relationships, and for other relationships where separate-query prefetching is appropriate.

```
departments = Department.objects.prefetch_related(
    "students"
)

for department in departments:
    for student in department.students.all():
        print(student.name)
```

Django fetches related results separately and joins them in Python.

Rule of thumb:

- `select_related()`: Single-valued relationships such as ForeignKey and OneToOne.
- `prefetch_related()`: Collections of related objects, such as ManyToMany and reverse ForeignKey.

Choose based on the relationship and query requirements; both methods have trade-offs.

Part 10: Aggregation and Annotation

26. aggregate()

`aggregate()` returns a dictionary containing summary values for a QuerySet.

```
from django.db.models import Avg, Count, Max, Min, Sum

Student.objects.aggregate(
    total=Count("id"),
    average_age=Avg("age"),
    maximum_age=Max("age"),
    minimum_age=Min("age")
)
```

27. annotate()

`annotate()` adds calculated values to each result object.

```
from django.db.models import Count
```

```
departments = Department.objects.annotate(
    student_count=Count("students")
)

for department in departments:
    print(department.name, department.student_count)
```

Difference:

- `aggregate()`: Summarizes the entire selected set.
- `annotate()`: Adds calculated values to individual rows or groups.

Be careful with joins and multiple aggregations, which can produce unexpected counts. Use `distinct=True` where appropriate and inspect the generated SQL.

Part 11: F Expressions

28. What Is F()?

F expressions refer to database field values without loading those values into Python first.

```
from django.db.models import F

Student.objects.filter(id=1).update(
    age=F("age") + 1
)
```

This performs the increment in the database.

Benefits:

- Avoids some read-modify-write race conditions.
- Enables calculations between database fields.
- Reduces unnecessary database round trips.

F expressions do not replace transactions or row locking for every concurrency-sensitive operation.

Part 12: Conditional Expressions and Database Functions

29. Case and When

```
from django.db.models import Case, When, Value, CharField

students = Student.objects.annotate(
    age_group=Case(
```

```
        When(age__lt=18, then=Value("Minor")),
        default=Value("Adult"),
        output_field=CharField()
    )
)
```

30. Database Functions

Examples include:

```
from django.db.models.functions import Lower, Upper, Length

students = Student.objects.annotate(
    lowercase_name=Lower("name"),
    uppercase_name=Upper("name"),
    name_length=Length("name")
)
```

Available functions and exact behavior can vary by Django version and database backend. Check the Django 6.0 database function reference.

Part 13: Subqueries and Exists

31. Exists()

Use `Exists()` to test whether a related query contains a matching row.

```
from django.db.models import Exists, OuterRef

active_courses = Course.objects.filter(
    students=OuterRef("pk")
)

students = Student.objects.annotate(
    has_active_course=Exists(active_courses)
)
```

This example checks for a related course, but the model would need an appropriate definition of an active course for the name to be meaningful.

32. Subquery()

Use `Subquery()` to embed the result of another query in a larger query.

```
from django.db.models import OuterRef, Subquery

# Example models: Blog and Entry, where Entry has
```

```
# a ForeignKey named blog and a pub_date field.

latest_entry = Entry.objects.filter(
    blog=OuterRef("pk")
).order_by("-pub_date")

blogs = Blog.objects.annotate(
    latest_entry_date=Subquery(
        latest_entry.values("pub_date")[:1]
    )
)
```

Subqueries are useful for correlated lookups, latest related records, and other advanced queries.

Part 14: Managers and Custom QuerySets

33. What Is a Manager?

A model manager provides the interface for database queries. The default manager is commonly accessed through `Model.objects`.

34. Custom QuerySet

```
class PublishedQuerySet(models.QuerySet):
    def published(self):
        return self.filter(status="published")


class Article(models.Model):
    title = models.CharField(max_length=200)
    status = models.CharField(max_length=20)

    objects = PublishedQuerySet.as_manager()
```

Usage:

```
Article.objects.published()
```

Custom managers and QuerySets help organize reusable query logic. Be aware that a filtered default manager can affect framework operations and related-object access, so choose default-manager behavior carefully.

Part 15: Transactions

35. What Is a Database Transaction?

A transaction groups database operations so they can be committed together or rolled back together.

36. transaction.atomic()

```
from django.db import transaction

with transaction.atomic():
    # Perform related database operations here.
    pass
```

If an exception causes the transaction to roll back, the enclosed database changes are rolled back as appropriate.

37. transaction.on_commit()

Use `on_commit()` to schedule a callback after a successful transaction commit.

```
from django.db import transaction

with transaction.atomic():
    # Save database records here.
    transaction.on_commit(
        lambda: print("Transaction committed")
    )
```

The callback does not run if the transaction is rolled back.

38. select_for_update()

For supported databases, `select_for_update()` locks selected rows until the transaction finishes.

```
from django.db import transaction

with transaction.atomic():
    student = Student.objects.select_for_update().get(
        id=1
    )
    # Perform concurrency-sensitive work here.
```

Locking behavior differs across database backends. Use an appropriate transaction and test the behavior against the production database.

Part 16: Raw SQL

39. Executing Raw SQL

Django ORM should be preferred when it can express the query clearly.

For specialized queries, Django supports raw SQL.

Example using a cursor:

```
from django.db import connection

with connection.cursor() as cursor:
    cursor.execute(
        "SELECT id, name FROM students WHERE age > %s",
        [18]
    )
    rows = cursor.fetchall()
```

Always parameterize values. Do not build SQL by concatenating user input into query strings.

Model-based raw queries are also available through methods such as `Model.objects.raw()`. Read the Django 6.0 documentation before using advanced raw SQL features.

Part 17: Database Optimization

40. Avoid N+1 Queries

An N+1 query problem happens when one query retrieves a list of objects and additional queries retrieve related data for each object.

Potentially inefficient:

```
students = Student.objects.all()

for student in students:
    print(student.department.name)
```

Improved:

```
students = Student.objects.select_related(
    "department"
)

for student in students:
    print(student.department.name)
```

41. Use Only Required Fields

```
Student.objects.only("id", "name")

Student.objects.defer("large_description")
```

Use these methods only when appropriate. Accessing deferred fields later may trigger additional queries.

42. Use values() and values_list()

```
Student.objects.values("name", "email")

Student.objects.values_list("name", flat=True)
```

These can reduce the overhead of constructing model instances when only field values are required.

43. Use exists() and count() Appropriately

Check for records:

```
Student.objects.filter(age=20).exists()
```

Count matching records:

```
Student.objects.filter(age=20).count()
```

Do not fetch all objects just to determine whether any exist or how many match.

44. Indexes

An index can improve filtering, ordering, and joining for suitable queries.

```
class Student(models.Model):
    email = models.EmailField(unique=True)
    age = models.PositiveIntegerField(db_index=True)
```

Indexes also consume storage and can slow writes. Add indexes based on real query patterns and measurement rather than indexing every field.

45. Inspect SQL and Query Plans

Inspect SQL generated by a QuerySet:

```
queryset = Student.objects.filter(age__gte=18)
print(queryset.query)
```

Inspect a query plan:

```
print(queryset.explain())
```

The exact query plan output and supported options depend on the database backend.

Other best practices:

- Use pagination for large result sets.
- Avoid loading huge QuerySets into memory unnecessarily.
- Measure database query counts and timings.
- Use Django Debug Toolbar or suitable database monitoring tools during development.
- Test performance with realistic data volumes.
- Use `iterator()` when appropriate for streaming through large QuerySets.

Part 18: Bulk Operations

46. bulk_create()

```
Student.objects.bulk_create([
    Student(name="Omar", email="omar@example.com", age=22),
    Student(name="Rahim", email="rahim@example.com", age=23),
])
```

47. bulk_update()

```
students = list(Student.objects.filter(age=22))

for student in students:
    student.age = 23

Student.objects.bulk_update(
    students,
    ["age"]
)
```

Bulk methods can substantially reduce database queries, but they do not execute the normal per-instance `save()` lifecycle or emit ordinary save signals. Review the Django 6.0 documentation for limitations involving primary keys, conflicts, and database support.

Part 19: Multiple Databases

48. Database Routing

Django can be configured to use multiple databases.

Example:

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "default.sqlite3",
    },
    "analytics": {
        "ENGINE": "django.db.backends.sqlite3",
        "NAME": BASE_DIR / "analytics.sqlite3",
    },
}
```

Choose a database explicitly:

```
Student.objects.using("analytics").all()
```

For advanced setups, Django supports database routers.

Important:

- Cross-database joins are generally not supported by Django's ORM.
- ForeignKey and ManyToMany relationships do not provide ordinary cross-database referential integrity.
- Transactions across multiple database connections are not automatically one atomic transaction.

Part 20: Fixtures and Data Management

49. Fixtures

Fixtures are files containing serialized data that can be loaded into the database.

Export data:

```
python manage.py dumpdata students --indent 2
```

Load fixture data:

```
python manage.py loaddata students.json
```

Use fixtures for suitable test data, sample datasets, and controlled data-loading workflows.

Review sensitive information before exporting data. Avoid placing real production secrets or personal data in committed fixtures.

50. Database Reset and Backup

Do not treat migration reversal, flushing, or database deletion as a substitute for a tested backup strategy.

Before destructive database operations:

- Confirm the database environment.
- Back up important data.
- Understand which tables and records will be affected.
- Test restoration procedures.

Part 21: Signals and Model Save Behavior

51. Common Signals

Examples include:

- `pre_save`
- `post_save`
- `pre_delete`
- `post_delete`
- `m2m_changed`

Signals allow code to react to certain model lifecycle events.

Example:

```
from django.db.models.signals import post_save
from django.dispatch import receiver

@receiver(post_save, sender=Student)
def student_saved(sender, instance, created, **kwargs):
    if created:
        print("A student was created")
```

Signals should be registered reliably, usually through an application's `AppConfig.ready()` method or an equivalent supported setup.

Important:

- `QuerySet.update()` does not call each instance's `save()` method.
- Bulk creation and bulk updates do not emit normal per-instance save signals.
- Signals can make program flow harder to follow.
- For many workflows, explicit service functions are clearer than signals.

Part 22: Database Integrity and Security

52. Constraints

Database constraints protect data integrity at the database level.

Examples:

- `UniqueConstraint`
- `CheckConstraint`

Use model validation for useful error messages, but do not rely on validation alone when a rule must remain true under concurrent database operations. Database constraints are important for enforcing uniqueness and other integrity rules.

53. SQL Injection Prevention

Safe ORM query:

```
Student.objects.filter(name=user_input)
```

Avoid constructing SQL with string concatenation:

```
# Unsafe pattern — do not use:
# query = "SELECT * FROM students WHERE name = '" + user_input + "'"
```

When using raw SQL, pass values as parameters rather than interpolating them into SQL strings.

54. Validation

A model's `save()` method does not automatically call `full_clean()`.

For explicit model validation:

```
student = Student(
    name="Omar",
    email="omar@example.com",
    age=22
)
student.full_clean()
student.save()
```

Validation, uniqueness checks, and database constraints have different purposes. Database constraints remain essential for protecting integrity under concurrency.

Part 23: Common Interview Questions

55. What is Django ORM?

Django ORM maps Python model classes to relational database tables and provides a Python API for querying and modifying data.

56. What is a QuerySet?

A QuerySet represents a collection of objects matching a database query. It is usually lazy until evaluated.

57. What is the difference between get() and filter()?

`get()` retrieves exactly one object and raises exceptions if zero or multiple objects match. `filter()` returns a QuerySet containing all matching objects.

58. What is the difference between null=True and blank=True?

`null=True` allows SQL NULL at the database level. `blank=True` allows an empty value during validation.

59. What is the difference between select_related() and prefetch_related()?

`select_related()` uses SQL joins for single-valued relationships. `prefetch_related()` fetches related collections separately and combines them in Python.

60. What is the difference between annotate() and aggregate()?

`annotate()` adds calculated values to individual objects or groups. `aggregate()` returns summary values for the selected set.

61. What is an F expression?

An F expression refers to a database field value within a query, allowing calculations in the database without first retrieving the value into Python.

62. What is a Q object?

A Q object allows complex query conditions using AND, OR, and NOT logic.

63. What is a migration?

A migration records and applies changes to database schema over time.

64. What is transaction.atomic()?

It defines a transaction block in which database operations can be committed together or rolled back together.

65. What is an index?

An index is a database structure that can speed up certain queries at the cost of storage and additional write overhead.

66. What is the N+1 query problem?

It occurs when an initial query retrieves objects and additional queries are executed for each object's related data. Techniques such as `select_related()` and

`prefetch_related()` can help prevent it.

67. What is the difference between update() and save()?

`save()` persists an individual model instance and follows the normal save lifecycle. QuerySet `update()` directly updates matching rows and does not call each object's `save()` method or emit normal save signals.

68. What is the purpose of a ForeignKey?

A ForeignKey defines a many-to-one relationship between models.

69. What is the purpose of a ManyToManyField?

A ManyToManyField defines a relationship in which multiple records on either side can be associated with multiple records on the other side.

70. Why should raw SQL use parameters?

Parameterized SQL helps prevent SQL injection and ensures values are handled appropriately by the database driver.

Part 24: Quick Lookup Cheat Sheet

Requirement	Django ORM
Retrieve all	`Student.objects.all()`
Exact match	`Student.objects.filter(age=20)`
Greater than	`Student.objects.filter(age__gt=20)`
Greater than or equal	`Student.objects.filter(age__gte=20)`

Less than	`Student.objects.filter(age__lt=20)`
Match a list	`Student.objects.filter(age__in=[18, 20])`
Case-insensitive match	`Student.objects.filter(name__iexact="omar")`
Check existence	`Student.objects.filter(age=20).exists()`
Count records	`Student.objects.count()`
Sort ascending	`Student.objects.order_by("name")`
Sort descending	`Student.objects.order_by("-age")`

Return dictionaries	`Student.objects.values("name", "email")`
Return a single field	`Student.objects.values_list("name", flat=True)`
Increment field	`Student.objects.update(age=F("age") + 1)`
Delete matching records	`Student.objects.filter(age__lt=18).delete()`
Load related ForeignKey	`Student.objects.select_related("department")`
Load related collection	`Department.objects.prefetch_related("students")`

Part 25: Recommended Learning Roadmap

Beginner

1. Understand databases, tables, rows, and columns.
2. Learn Django models and common fields.
3. Configure SQLite or PostgreSQL.
4. Practice migrations.
5. Master CRUD operations.
6. Learn QuerySets and common lookups.

Intermediate

7. Practice ForeignKey, OneToOneField, and ManyToManyField.
8. Learn Q objects and F expressions.
9. Understand aggregation and annotation.
10. Practice `select_related()` and `prefetch_related()`.
11. Learn model validation and database constraints.
12. Write tests for database behavior.

Advanced

13. Learn transactions and concurrency.
14. Study indexes and query plans.
15. Practice subqueries and conditional expressions.
16. Learn custom QuerySets and managers.
17. Understand bulk operations and signals.
18. Explore raw SQL when necessary.
19. Learn database routing and multi-database limitations.
20. Measure and optimize queries using realistic workloads.

Official Django 6.0 Documentation

- Models: https://docs.djangoproject.com/en/6.0/topics/db/models/
- Making queries: https://docs.djangoproject.com/en/6.0/topics/db/queries/
- Aggregation: https://docs.djangoproject.com/en/6.0/topics/db/aggregation/
- Database search: https://docs.djangoproject.com/en/6.0/topics/db/search/
- Managers: https://docs.djangoproject.com/en/6.0/topics/db/managers/
- Raw SQL: https://docs.djangoproject.com/en/6.0/topics/db/sql/
- Transactions: https://docs.djangoproject.com/en/6.0/topics/db/transactions/

- Optimization: https://docs.djangoproject.com/en/6.0/topics/db/optimization/
- Instrumentation: https://docs.djangoproject.com/en/6.0/topics/db/instrumentation/
- Multiple databases: https://docs.djangoproject.com/en/6.0/topics/db/multi-db/
- Fixtures: https://docs.djangoproject.com/en/6.0/topics/db/fixtures/

Final Advice: Learn the ORM by building a small project, such as a Student Management System, Blog API, or Course Enrollment API. Practice each concept with real models, inspect generated SQL, and test your queries against your chosen database backend.